MARCH 17, 2023

ΜΕΤΑΒΛΗΤΕΣ JAVASCRIPT

ΡΟΥΠΑ ΒΑΣΙΛΙΚΗ
ΙΕΚ ΔΕΛΤΑ 360

ΠΕΡΙΕΧΟΜΕΝΑ

1 Μεταβλητές JavaScript 2
1.1 Τι είναι οι Μεταβλητές; 2
2 ορισμοσ και χαρακτηριστικα μεταβλητων στην JavaScript 2
2.1 Είδη μεταβλητών 2
2.1.1 Ορισμός μεταβλητών με την χρήση let 3
2.1.2 Block Scope-Πεδίο εφαρμογής σε τμήμα κώδικα 4
2.1.3 Global Scope-Καθολικό Πεδίο Εφαρμογής 4
2.1.4 Δεν μπορεί να επαναδηλωθεί 4
2.1.5 Εκ νέου δήλωση μεταβλητών 5
2.1.6 JavaScript Const 6
2.1.7 Πότε να χρησιμοποιήσετε το JavaScript const; 6
2.1.8 Παράδειγμα με χρήση const 7
2.1.9 Μικτό παράδειγμα 7
2.1.10 Πότε να χρησιμοποιήσετε τις var, let ή const; 7
2.2 Ορισμός ονομάτων μεταβλητών - Αναγνωριστικά JavaScript 8
2.3 Ο τελεστής ανάθεσης 8
2.4 Τύποι δεδομένων JavaScript 8
2.4.1 Είδη Τύπων Δεδομένων-Data Types 9
2.5 Δήλωση μιας μεταβλητής JavaScript 10
2.6 Μία δήλωση, πολλές μεταβλητές 10
2.7 Τιμή = undefined 11
2.8 Επαναπροσδιορισμός μεταβλητών JavaScript 11

1 ΜΕΤΑΒΛΗΤΕΣ JAVASCRIPT

1.1 ΤΙ ΕΙΝΑΙ ΟΙ ΜΕΤΑΒΛΗΤΕΣ;

Στα μαθηματικά, το **γράμμα** ή το **σύμβολο** που χρησιμοποιείται σε μια αριθμητική παράσταση στη θέση μιας τιμής άγνωστης ή μεταβαλλόμενης λέγεται **μεταβλητή**.

Στον προγραμματισμό, οι μεταβλητές είναι δεξαμενές αποθήκευσης δεδομένων (αποθήκευση τιμών δεδομένων).

Σε αυτό το παράδειγμα, τα x, y και z είναι μεταβλητές που δηλώνονται με τη λέξη-κλειδί var:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Μεταβλητές</h1>

<p>Σε αυτό το παράδειγμα, τα x, y και z είναι μεταβλητές</p>

<p id="demo"></p>

<script>
var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML = z;
</script>

</body>
</html>
```

JavaScript Μεταβλητές

Σε αυτό το παράδειγμα, τα x, y και z είναι μεταβλητές

11

Από το παραπάνω παραδείγματα, διαπιστώνουμε οτι το x αποθηκεύει την τιμή 5, το y αποθηκεύει την τιμή 6, το z αποθηκεύει την τιμή 11

2 ΟΡΙΣΜΟΣ ΚΑΙ ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ ΜΕΤΑΒΛΗΤΩΝ ΣΤΗΝ JAVASCRIPT

2.1 ΕΙΔΗ ΜΕΤΑΒΛΗΤΩΝ

Οι μεταβλητές είναι δοχεία αποθήκευσης δεδομένων

Οι μεταβλητές JavaScript μπορούν να δηλωθούν με 4 τρόπους:

α. Αυτόματα
β. Χρησιμοποιώντας var
χ. Χρησιμοποιώντας let
δ. Χρήση const

Σε αυτό το πρώτο παράδειγμα, οι x, y και z είναι μη δηλωμένες μεταβλητές. Δηλώνονται αυτόματα όταν χρησιμοποιούνται για πρώτη φορά:

ΜΕΤΑΒΛΗΤΕΣ JAVASCRIPT

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are undeclared.</p>
<p>They are automatically declared when first used.</p>

<p id="demo"></p>

<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, x, y, and z are undeclared.

They are automatically declared when first used.

The value of z is: 11

Σημείωση: Θεωρείται καλή πρακτική προγραμματισμού να δηλώνετε πάντα τις μεταβλητές πριν τις χρησιμοποιήσετε.

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

Σημείωση

α. Η λέξη-κλειδί var χρησιμοποιήθηκε στην JavaScript από το 1995 έως το 2015.
β. Οι λέξεις-κλειδιά let και const προστέθηκαν στη JavaScript το 2015.
χ. Η λέξη-κλειδί var θα πρέπει να χρησιμοποιείται μόνο σε κώδικα γραμμένο για παλαιότερους φυλλομετρητές.

2.1.1 Ορισμός μεταβλητών με την χρήση let

Χαρακτηριστικά ορισμού μεταβλητών με την χρήση let

Οι μεταβλητές που δηλώνονται με let έχουν εφαρμογή σε συγκεκριμένο πεδίο δράσης ενός τμήματος κώδικα(block scope)
Οι μεταβλητές που δηλώνονται με let πρέπει να δηλωθούν πριν από τη χρήση τους
Οι μεταβλητές που δηλώνονται με let δεν μπορούν να επαναδηλωθούν στο ίδιο πεδίο δράσης κώδικα
Σημείωση: η συγκεκριμένη έκταση τμήματος block ή πεδίο δράσης τμήματος κώδικα ή block scope είναι το τμήμα κώδικα που ορίζεται από ένα ζευγάρι αγκυλών {}.

2.1.2 Block Scope-Πεδίο εφαρμογής σε τμήμα κώδικα

Πριν από την ES6 (2015), η JavaScript δεν είχε Block Scope.
Η JavaScript είχε Global Scope(Καθολική Εφαρμογή) και Function Scope (εφαρμογή σε συναρτήσεις).
Η ES6 εισήγαγε τις δύο νέες λέξεις-κλειδιά της JavaScript: let και const.
Αυτές οι δύο λέξεις-κλειδιά ορισμού μεταβλητών παρέχουν εφαρμογή σε τμήμα κώδικα (Block Scope) στη JavaScript:

Δεν είναι δυνατή η πρόσβαση στις μεταβλητές που δηλώνονται μέσα σε ένα τμήμα κώδικα { }:

```
{
  let x = 2;
}
// x can NOT be used here
```

2.1.3 Global Scope-Καθολικό Πεδίο Εφαρμογής

Οι μεταβλητές που δηλώνονται με το `var` πάντα έχουν **καθολικό πεδίο εφαρμογής**

Οι μεταβλητές που δηλώνονται με τη `var` λέξη-κλειδί ΔΕΝ μπορούν να εφαρμοστούν σε συγκεκριμένο τμήμα πεδίου, αυτές είναι προσβάσιμες και εκτός του τμήματος κώδικα:

```
{
  var x = 2;
}
// x CAN be used here
```

2.1.4 Δεν μπορεί να επαναδηλωθεί

Οι μεταβλητές που ορίζονται με `let` **δεν μπορούν** να δηλωθούν εκ νέου.

Δεν μπορείτε να επαναδηλώσετε μια μεταβλητή που έχει δηλωθεί με `let`.

Δηλαδή **δεν μπορείτε** να κάνετε αυτό:

```
let x = "John Doe";

let x = 0;
```

Ενώ, οι μεταβλητές που ορίζονται με `var` **μπορούν** να δηλωθούν εκ νέου.

Δηλαδή **μπορείτε** να κάνετε αυτό:

```
var x = "John Doe";

var x = 0;
```

2.1.5 Εκ νέου δήλωση μεταβλητών

Η εκ νέου δήλωση μιας μεταβλητής χρησιμοποιώντας τη `var` λέξη-κλειδί μπορεί να δημιουργήσει προβλήματα. Δηλαδή,η επαναδήλωση μιας μεταβλητής μέσα σε ένα τμήμα κώδικα θα κάνει αυτήν την μεταβλητή να υπακούει στην τιμή του τμήματος κώδικα της και έξω από αυτή.

```
var x = 10;
// Here x is 10

{
var x = 2;
// Here x is 2
}

// Here x is 2
```

Η εκ νέου δήλωση μιας μεταβλητής χρησιμοποιώντας τη `let` λέξη-κλειδί μπορεί να λύσει αυτό το πρόβλημα.

Η εκ νέου δήλωση μιας μεταβλητής μέσα σε ένα μπλοκ δεν θα επαναδηλώσει τη μεταβλητή εκτός του μπλοκ:

```
let x = 10;
// Here x is 10

{
let x = 2;
// Here x is 2
}

// Here x is 10
```

2.1.5.1 Παράδειγμα με χρήση let

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
let x = 5;
let y = 6;
let z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

2.1.6 JavaScript Const

Οι μεταβλητές που ορίζονται με `const` δεν μπορούν να **επαναδηλωθούν**

Οι μεταβλητές που ορίζονται με `const` δεν μπορούν να **αντιστοιχιστούν εκ νέου**

Οι μεταβλητές που ορίζονται με `const` έχουν εφαρμογή σε συγκεκριμένο τμήμα κώδικα **Block Scope**

Δεν είναι δυνατή η εκ νέου αντιστοίχιση μιας μεταβλητής που ορίζεται με τη `const` λέξη-κλειδί

```
const PI = 3.141592653589793;
PI = 3.14;      // This will give an error
PI = PI + 10;   // This will also give an error
```

Στις μεταβλητές `const` πρέπει να εκχωρηθεί μια τιμή όταν δηλώνονται:

Σωστή σύνταξη

```
const PI = 3.14159265359;
```

Λάθος σύνταξη

```
const PI;
PI = 3.14159265359;
```

2.1.7 Πότε να χρησιμοποιήσετε το JavaScript const;

Πάντα να δηλώνετε μια μεταβλητή με `const` όταν γνωρίζετε ότι η τιμή δεν πρέπει να αλλάξει.

Χρησιμοποιήστε `const` όταν δηλώνετε:

- Έναν νέο πίνακα(array)
- Ένα νέο αντικείμενο(object)
- Μια νέα συνάρτηση (function)
- Ένα νέο RegExp (regular expression)

Η λέξη-κλειδί `const` είναι λίγο παραπλανητική. Δεν ορίζει σταθερή τιμή. Ορίζει μια σταθερή αναφορά σε μια τιμή.

Εξαιτίας αυτού ΔΕΝ μπορείτε:

- Να εκχωρήσετε ξανά μια σταθερή τιμή.
- Να αναθέσετε τιμές ξανά σε έναν πίνακα δηλωμένο με const.
- Να αναθέσετε τιμές ξανά σε ένα αντικειμένο δηλωμένο με const.

Αλλά μπορείτε:

- Να αλλάξετε τα στοιχεία του πίνακα

- Να αλλάξετε τις ιδιότητες του αντικειμένου

2.1.8 Παράδειγμα με χρήση const

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
const x = 5;
const y = 6;
const z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

2.1.9 Μικτό παράδειγμα

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Variables</h1>

<p>In this example, price1, price2, and total are variables.</p>

<p id="demo"></p>

<script>
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML =
"The total is: " + total;
</script>

</body>
</html>
```

JavaScript Variables

In this example, price1, price2, and total are variables.

The total is: 11

Οι δύο μεταβλητές price1 και price2 δηλώνονται με τη λέξη κλειδί const. Πρόκειται για σταθερές τιμές και δεν μπορούν να αλλάξουν.Η μεταβλητή total δηλώνεται με τη λέξη κλειδί let.Η τιμή total μπορεί να αλλάξει.

2.1.10 Πότε να χρησιμοποιήσετε τις var, let ή const;

Να δηλώνετε πάντα μεταβλητές

1. Χρησιμοποιείτε πάντα const αν η τιμή δεν πρέπει να αλλάξει
2. Χρησιμοποιείτε πάντα const αν ο τύπος δεν πρέπει να αλλάξει (πίνακες και αντικείμενα).
3. Χρησιμοποιήστε let μόνο αν δεν μπορείτε να χρησιμοποιήσετε const

4. Χρησιμοποιήστε var μόνο αν ΠΡΕΠΕΙ να υποστηρίζετε παλαιά προγράμματα περιήγησης.

2.2 ΟΡΙΣΜΟΣ ΟΝΟΜΑΤΩΝ ΜΕΤΑΒΛΗΤΩΝ - ΑΝΑΓΝΩΡΙΣΤΙΚΑ JAVASCRIPT

Όλες οι μεταβλητές JavaScript πρέπει να προσδιορίζονται με μοναδικά ονόματα.Αυτά τα μοναδικά ονόματα ονομάζονται αναγνωριστικά.

Τα αναγνωριστικά μπορεί να είναι σύντομα ονόματα (όπως x και y) ή περισσότερα περιγραφικά ονόματα (ηλικία, άθροισμα, συνολικός όγκος).

Οι γενικοί κανόνες για την κατασκευή ονομάτων για μεταβλητές (μοναδικά αναγνωριστικά) είναι:

- Τα ονόματα μπορεί να περιέχουν γράμματα, ψηφία, κάτω παύλες και σύμβολα δολαρίου.
- Τα ονόματα πρέπει να ξεκινούν με ένα γράμμα.
- Τα ονόματα μπορούν επίσης να ξεκινούν με $ και _
- Τα ονόματα κάνουν διάκριση πεζών-κεφαλαίων (y και Y είναι διαφορετικές μεταβλητές).
- Οι δεσμευμένες λέξεις (όπως var,let, const, function, switch,if) δεν μπορούν να χρησιμοποιηθούν ως ονόματα.

2.3 Ο ΤΕΛΕΣΤΗΣ ΑΝΑΘΕΣΗΣ

Στη JavaScript, το σύμβολο ισότητας (=) είναι ένας τελεστής "ανάθεσης" και όχι ένας τελεστής "ίσο με".Αυτό είναι διαφορετικό από την άλγεβρα.
Το παρακάτω δεν έχει νόημα στην άλγεβρα: x = x + 5
Στη JavaScript, ωστόσο, έχει απόλυτο νόημα: αναθέτει την τιμή x + 5 στο x.
(Υπολογίζει την τιμή του x + 5 και τοποθετεί το αποτέλεσμα στο x. Η τιμή του x αυξάνεται κατά 5).

Σημείωση: Ο τελεστής "ίσο με" γράφεται ως == στη JavaScript.

2.4 ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ JAVASCRIPT

Οι μεταβλητές JavaScript μπορούν να περιέχουν αριθμούς όπως το 100 και τιμές κειμένου όπως "Επεξεργασία".

Στον προγραμματισμό, οι τιμές κειμένου ονομάζονται συμβολοσειρές κειμένου/strings.

Η JavaScript μπορεί να χειριστεί πολλούς τύπους δεδομένων, αλλά προς το παρόν, σκεφτείτε απλώς αριθμούς και συμβολοσειρές.

Οι συμβολοσειρές γράφονται σε διπλά ή μονά εισαγωγικά. Οι αριθμοί γράφονται χωρίς εισαγωγικά.

Εάν βάλετε έναν αριθμό σε εισαγωγικά, θα αντιμετωπιστεί ως κείμενο και όχι ως αριθμός.

```
const pi = 3.14;
let person = "John Doe";
let answer = 'Yes I am!';
```

ΜΕΤΑΒΛΗΤΕΣ JAVASCRIPT

2.4.1 Είδη Τύπων Δεδομένων-Data Types

Οι τύποι δεδομένων είναι οι ταξινομήσεις που δίνουμε στα διαφορετικά είδη δεδομένων που χρησιμοποιούμε στον προγραμματισμό. Στη JavaScript, υπάρχουν επτά βασικοί τύποι δεδομένων:

1. Number: Οποιοσδήποτε αριθμός, συμπεριλαμβανομένων των αριθμών με δεκαδικούς αριθμούς: 4, 8, 1516, 23,42.Οι αριθμοί γράφονται με ή χωρίς δεκαδικούς και χωρις μονά ή διπλά εισαγωγικά
2. 10.5
3. Strings-Συμβολοσειρά: Οποιαδήποτε ομαδοποίηση χαρακτήρων στο πληκτρολόγιό σας (γράμματα, αριθμοί, κενά, σύμβολα κ.λπ.) που περιβάλλεται από μονά εισαγωγικά: ' ... ' ή διπλά εισαγωγικά " ... ", αν και προτιμούμε τα μονά εισαγωγικά. Μερικοί άνθρωποι θέλουν να σκέφτονται τη συμβολοσειρά ως μια φανταχτερή λέξη για κείμενο.Οι συμβολοσειρές/strings είναι κείμενο, γραμμένο σε διπλά ή μονά εισαγωγικά:πχ var x ="Javascript";
4. Boolean: Αυτός ο τύπος δεδομένων έχει μόνο δύο πιθανές τιμές — είτε true είτε false (χωρίς εισαγωγικά). Είναι χρήσιμο να σκεφτόμαστε τα boolean ως διακόπτες ενεργοποίησης και απενεργοποίησης ή ως απαντήσεις σε μια ερώτηση «ναι» ή «όχι».
5. Null: Αυτός ο τύπος δεδομένων αντιπροσωπεύει την σκόπιμη απουσία μιας τιμής και αντιπροσωπεύεται από τη λέξη-κλειδί null (χωρίς εισαγωγικά).
6. Undefined: Αυτός ο τύπος δεδομένων υποδηλώνεται με τη λέξη-κλειδί undefined (χωρίς εισαγωγικά). Αντιπροσωπεύει επίσης την απουσία τιμής αν και έχει διαφορετική χρήση από τη μηδενική. Undefined-απροσδιόριστο σημαίνει ότι μια δεδομένη τιμή δεν υπάρχει.
7. Symbol-Σύμβολο: Ένα νεότερο χαρακτηριστικό της γλώσσας, τα σύμβολα είναι μοναδικά αναγνωριστικά, χρήσιμα σε πιο σύνθετη κωδικοποίηση. Δεν χρειάζεται να ανησυχείτε για αυτά προς το παρόν.
8. object: Συλλογές σχετικών δεδομένων.

Οι πρώτοι 6 από αυτούς τους τύπους ονομάζονται πρωτόγονοι (primitive) τύποι δεδομένων. Είναι οι πιο βασικοί τύποι δεδομένων στη γλώσσα. Τα αντικείμενα είναι πιο περίπλοκα και θα μάθετε πολύ περισσότερα για αυτά καθώς προχωράτε μέσω της JavaScript, το συγκεκριμένο είδος δεδομένων ονομάζεται μη-πρωτόγονοι (non-primitive). Οι επτά τύποι μπορεί να μην φαίνονται τόσο πολλοί, αλλά σύντομα θα παρατηρήσετε ότι ο κόσμος ανοίγει με δυνατότητες μόλις αρχίσετε να αξιοποιείτε τον καθένα. Καθώς μαθαίνετε περισσότερα για τα αντικείμενα, θα μπορείτε να δημιουργείτε πολύπλοκες συλλογές δεδομένων.

Η JavaScript χρησιμοποιεί αριθμητικούς τελεστές (+ - * /) για τον υπολογισμό τιμών και έναν τελεστή εκχώρησης/ανάθεσης (=) για να εκχωρήσει τιμές τόσο σε παραστάσεις όσο και σε μεταβλητές: (5 + 6) * 10

2.5 ΔΗΛΩΣΗ ΜΙΑΣ ΜΕΤΑΒΛΗΤΗΣ JAVASCRIPT

Η δημιουργία μιας μεταβλητής σε JavaScript ονομάζεται "δήλωση/declaration" μιας μεταβλητής.

Δηλώνετε μια μεταβλητή JavaScript με τη λέξη-κλειδί var ή let :

let carName;

Μετά τη δήλωση, η μεταβλητή δεν έχει τιμή (τεχνικά είναι απροσδιόριστη).Για να αντιστοιχίσετε μια τιμή στη μεταβλητή, χρησιμοποιήστε το σύμβολο ίσον (=):

carName = "Volvo";

Μπορείτε εναλλακτικά να εκχωρήσετε/ορίσετε μια τιμή στη μεταβλητή όταν τη δηλώνετε:

let carName = "Volvo";

Στο παρακάτω παράδειγμα, δημιουργούμε μια μεταβλητή που ονομάζεται carName και της εκχωρούμε την τιμή "Volvo".

Στη συνέχεια "εξάγουμε" την τιμή μέσα σε μια παράγραφο HTML με id="demo" ώστε να την προβάλλουμε στην οθόνη:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript μεταβλητές</h1>

<p>Δημιουργήστε μια μεταβλητή, αντιστοιχίστε μια τιμή σε αυτήν και εμφανίστε
την:</p>

<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

JavaScript μεταβλητές

Δημιουργήστε μια μεταβλητή, αντιστοιχίστε μια τιμή σε αυτήν και εμφανίστε την:

Volvo

Είναι μια καλή πρακτική προγραμματισμού να δηλώνετε όλες τις μεταβλητές στην αρχή του κώδικα.

2.6 ΜΙΑ ΔΗΛΩΣΗ, ΠΟΛΛΕΣ ΜΕΤΑΒΛΗΤΕΣ

Μπορείτε να δηλώσετε πολλές μεταβλητές σε μία πρόταση. Ξεκινήστε τη δήλωση με let και διαχωρίστε τις μεταβλητές με κόμμα:

let person = "John Doe", carName = "Volvo", price = 200;

Μια δήλωση μπορεί να εκτείνεται σε πολλές γραμμές:

```
let person = "John Doe",
carName = "Volvo",
price = 200;
```

2.7 ΤΙΜΗ = UNDEFINED

Στα προγράμματα υπολογιστών, οι μεταβλητές δηλώνονται συχνά χωρίς τιμή. Η τιμή μπορεί να είναι κάτι που πρέπει να υπολογιστεί ή κάτι που θα παρασχεθεί αργότερα, όπως η εισαγωγή ενός στοιχείου μιας φόρμας απο έναν χρήστη.

Μια μεταβλητή που δηλώνεται χωρίς τιμή θα έχει την τιμή απροσδιόριστη (`undefined`).

Η μεταβλητή carName θα έχει την τιμή `undefined` μετά την εκτέλεση αυτής της δήλωσης:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Variables</h1>

<p>Μια μεταβλητή χωρίς τιμή έχει την τιμή:</p>
<p id="demo"></p>

<script>
var carName;
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

JavaScript Variables

Μια μεταβλητή χωρίς τιμή έχει την τιμή:

undefined

2.8 ΕΠΑΝΑΠΡΟΣΔΙΟΡΙΣΜΟΣ ΜΕΤΑΒΛΗΤΩΝ JAVASCRIPT

Εάν επαναδηλώσετε μια μεταβλητή JavaScript που έχει δηλωθεί με var, δεν θα χάσει την τιμή της.

Η μεταβλητή carName θα εξακολουθεί να έχει την τιμή "Volvo" μετά την εκτέλεση αυτών των δηλώσεων:

Παράδειγμα

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Variables</h1>

<p>If you re-declare a JavaScript variable, it will not lose its value.</p>

<p id="demo"></p>

<script>
var carName = "Volvo";
var carName;
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

JavaScript Variables

If you re-declare a JavaScript variable, it will not lose its value.

Volvo

Σημείωση: Δεν μπορείτε να επαναδηλώσετε μια μεταβλητή που έχει δηλωθεί με let ή const.

Αυτό δεν θα λειτουργήσει:

```
let carName = "Volvo";
let carName;
```